Shell lab

1.拉取tsh.c文件

使用vim打开tsh_helper.c/h文件查看实验帮助和指南。
根据六个目标函数的要求编写tsh.c文件并且编译

```
{
    char *argv[MAXARGS];/*Argument list execve() */
    char buf[MAXLINE];/*Holds modified command line */
    int bg;/*Should the job run in bg or fg? */
    pid_t pid;/*Process id */
    sigset_t mask_all,mask_one,prev_one;

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return;/* Ignore empty lines */

    if (!builtin_cmd(argv)) {
        //blocking SIGCHLD in if status,otherewise it maybe has bugs
        Sigfillset(&mask_all);/* add every signal number to set */
        Sigemptyset(&mask_one);/* create empty set */
        Sigaddset(&mask_one, SIGCHLD);/* add signal number to set */

        /*  block SIGINT and save previous blocked set */
        /* avoid parent process run to addjob exited,before fork child process block sigchild signal,after ca
block  */
        Sigprocmask(SIG_BLOCK, &mask_one, &prev_one); /* Block SIGCHLD */
        if ((pid = Fork()) == 0) {/* Child runs user job */
            /* restore previous blocked set,unblocking SIGINT */
            /* child process inherit parent process' blocking sets,avoid it can't receive itself child proces
we must unblock */
            Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
            Setpgid(0,0);// set child's group to a new process group (this is identical to the child's PID)
                                                                                                    198,1
        Sigemptyset(&mask_one);/* create empty set */
        Sigaddset(&mask_one, SIGCHLD);/* add signal number to set */

        /*  block SIGINT and save previous blocked set */
        /* avoid parent process run to addjob exited,before fork child process block sigchild signal,a
ock  */
        Sigprocmask(SIG_BLOCK, &mask_one, &prev_one); /* Block SIGCHLD */
        if ((pid = Fork()) == 0) {/* Child runs user job */
            /* restore previous blocked set,unblocking SIGINT */
            /* child process inherit parent process' blocking sets,avoid it can't receive itself child
 must unblock */
            Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
            Setpgid(0,0);// set child's group to a new process group (this is identical to the child's
            Execve(argv[0], argv, environ);//this function not return ,so must call exit,otherewise it
        }/* Parent waits for foreground job to terminate */

        Sigprocmask(SIG_BLOCK, &mask_all, NULL); /* Block SIGCHLD */
        int st = (bg==0) ? FG : BG;
        addjob(jobs,pid,st,cmdline);
        Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
        if (!bg) {
            //because sigchld_handler was called above, it call waitpid, so don't call and circular wa
            waitfg(pid);
        }
        else
            printf("[%d] (%d) %s", pid2jid(pid),pid, cmdline);
    }
    return;
```

```
int builtin_cmd(char **argv)
{
    if(!strcmp(argv[0],"quit")) /* quit command */
        exit(0);

    if (!strcmp(argv[0], "&")) /* Ignore singleton & */
        return 1;

    if(!strcmp((argv[0]),"jobs"))/* jobs command */
    {
        listjobs(jobs);
        return 1;
    }

    if(!strcmp((argv[0]),"fg") || !strcmp((argv[0]),"bg"))/* bg/fg command */
    {
        do_bgfg(argv);
        return 1;
    }
    return 0;     /* not a builtin command */
}


/*
 * do_bgfg - Execute the builtin bg and fg commands
 */
void do_bgfg(char **argv)
{
    if(!argv[1]){
        printf("%s command requires PID or %%jobid argument\n", argv[0]);
        return;
    }

    if (!isdigit(argv[1][0]) && argv[1][0] != '%') {            // Checks if the second argument is valid
        printf("%s: argument must be a PID or %%jobid\n", argv[0]);
        return;
    }

    struct job_t* myjob;
    if(argv[1][0]=='%'){//jid
        myjob = getjobjid(jobs,atoi(&argv[1][1]));
        if(!myjob){
            printf("%s: No such job\n", argv[1]);
            return;
        }
    }else{//pid
        myjob = getjobpid(jobs,atoi(argv[1]));
        if (!myjob) {                                   // Checks if the given PID is there
            printf("(%d): No such process\n", atoi(argv[1]));
            return;
        }
    }

    Kill(-myjob->pid,SIGCONT);//send continue signal
    if(!strcmp(argv[0],"bg")){
```

```
*/
void waitfg(pid_t pid)
{
    while(fgpid(jobs))
        usleep(1000);//sleep one second
    return;
}

/*****************
 * Signal handlers
 *****************/

/*
 * sigchld_handler - The kernel sends a SIGCHLD to the shell whenever
 *     a child job terminates (becomes a zombie), or stops because it
 *     received a SIGSTOP or SIGTSTP signal. The handler reaps all
 *     available zombie children, but doesn't wait for any other
 *     currently running children to terminate.
 */
void sigchld_handler(int sig)
{
    int olderrno = errno;
    sigset_t mask_all,prev;
    pid_t pid;
    int status;
    Sigfillset(&mask_all);
    while((pid = waitpid(-1,&status,WNOHANG | WUNTRACED))>0){
        // WNOHANG | WUNTRACED return immediately
        if (WIFEXITED(status))  // normally exited,delete job
```

370,1

```
*/
void sigint_handler(int sig)
{
    int olderrno = errno;
    pid_t fg = fgpid(jobs);
    if(fg){
        Kill(-fg,sig);
    }
    errno = olderrno;
    return;
}


/*
 * sigtstp_handler - The kernel sends a SIGTSTP to the shell whenever
 *     the user types ctrl-z at the keyboard. Catch it and suspend the
 *     foreground job by sending it a SIGTSTP.
 */
void sigtstp_handler(int sig)
{
    int olderrno = errno;
    pid_t fg = fgpid(jobs);
    if(fg){
        Kill(-fg,sig);
    }
    errno = olderrno;
    return;
```

2.根据课件完成关卡内容

2.1首先打开trace01.txt文件发现第一关调用了linux命令close

关闭文件并wait等待，在EOF上正常终止。使用sdriver验
证正确如下

```
Running traces/trace00.txt...
Success: The test and reference outputs for traces/trace00.txt matched!
Test output:
#
# trace00.txt - Properly terminate on EOF.
#


Reference output:
#
# trace00.txt - Properly terminate on EOF.
#



Summary: 1/1 correct iterations
```

2.2第二关需要我们针对输入的命令quit退出shell进程，我们

需要解析cmdline（输入的命令），判断是不是“quit”字符
串，是就退出 结果验证如下

```
bo@bo:~/shlab-handout$ make test02
./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
#
# trace02.txt - Process builtin quit command.
#
bo@bo:~/shlab-handout$ make rtest02
./sdriver.pl -t trace02.txt -s ./tshref -a "-p"
#
# trace02.txt - Process builtin quit command.
```

2.3打开trace03.txt发现第三关首先是： /bin/echo tsh> quit

意思是打开 bin 目录下的 echo 可执行文件，在
foregound 开启一个子进程运行它（因为末尾没有&符
号，如果有，就是在 backgound 运行）运行 echo 这个进
程的过程中，通过 tsh>quit 命令，调用 tsh并执行内置命

令 quit，退出 echo 这个子进程，最后在 tsh 中执行内置命令 quit，退出 tsh 进程，回到我们的终端。执行结果如下：

```
bo@bo:~/shlab-handout$ make test03
./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
bo@bo:~/shlab-handout$ make rtest03
./sdriver.pl -t trace03.txt -s ./tshref -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
```

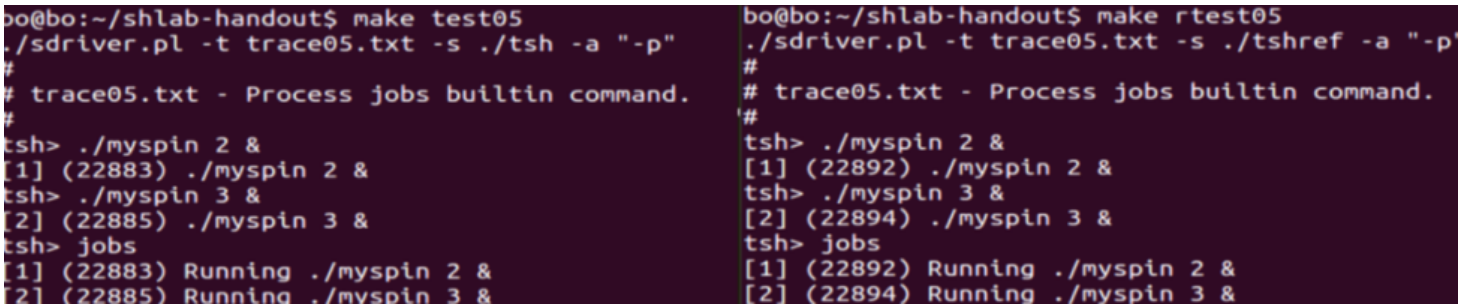
2.4第四关需要先在前台执行echo命令，等待程序执行完毕回

收子进程。&代表是一个后台程序，myspin睡眠1秒，然后停止。因为在后台，所以显示下面一句，如果在前台则无。执行结果如下：

```
bo@bo:~/shlab-handout$ make test04
./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (22859) ./myspin 1 &
bo@bo:~/shlab-handout$ make rtest04
./sdriver.pl -t trace04.txt -s ./tshref -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (22865) ./myspin 1 &
```

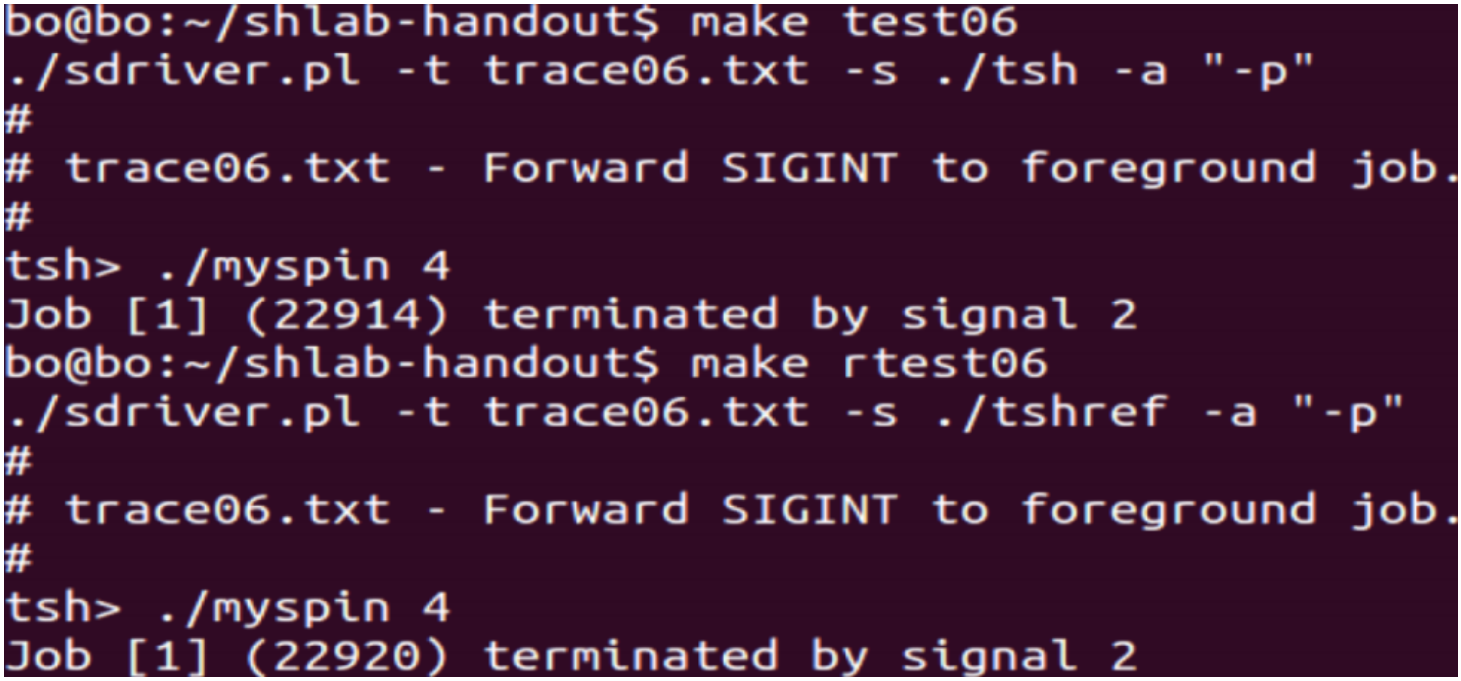
2.5第五关分别运行了前台echo、后台myspin、前台echo、

后台myspin，然后需要实现一个内置命令job，功能是显示目前任务列表中的所有任务的所有属性,运行结果如下



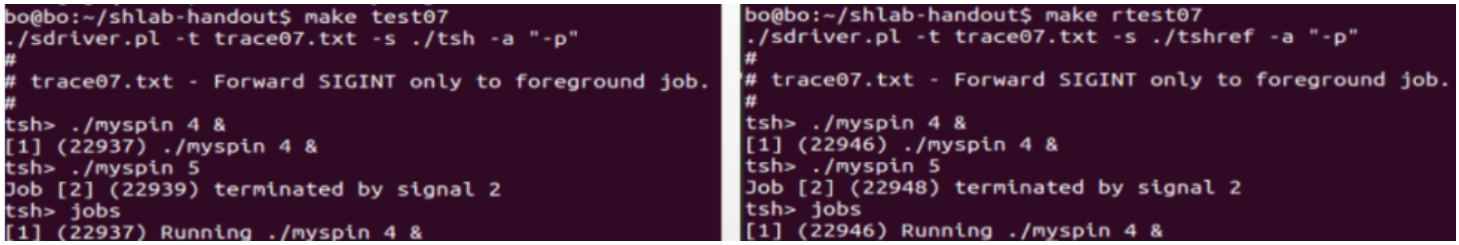
2.6第六关接收到了中断信号SIGINT（即CTRL_C）那么结束

前台进程。验证如下：



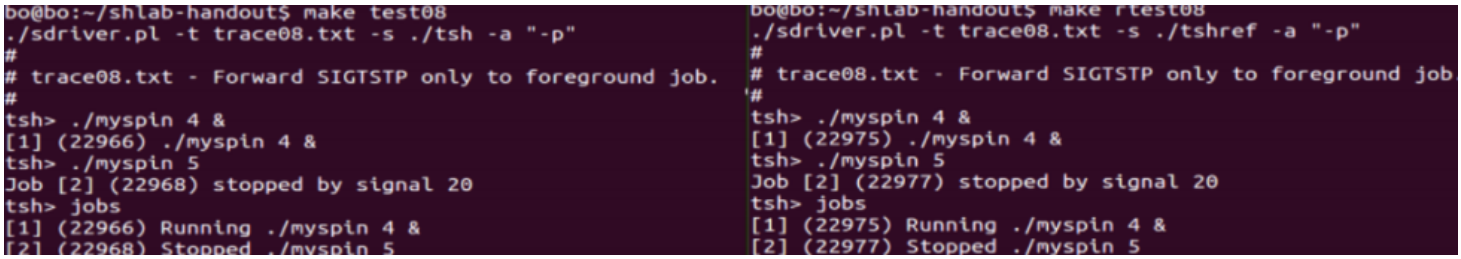
2.7打开trace07.txt 根据注释，可以知道第七关测试的是只将

SIGINT转发给前台作业。这里的命令行其实根据前面的就很好理解了，就是给出两个作业，一个在前台工作，另一个在后台工作，接下来传递SIGINT指令，然后调用内置指令jobs来查看此时的工作信息，来对比出是不是只将SIGINT转发给前台作业。结果如下



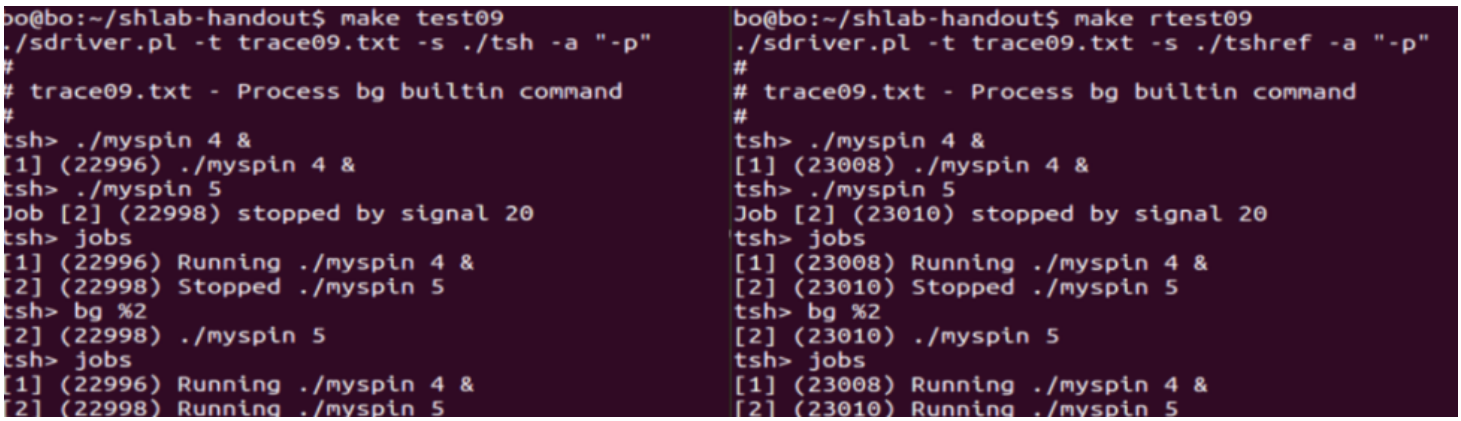
2.8根据注释是需要将SIGTSTP转发给前台作业。根

据这个信号的作用，也就是该进程会停止直到下一个SIGCONT也就是挂起，让别的程序继续运行。这里也就是运行了后台程序，然后使用jobs来打印出进程的信息 结果如下



2.9trace09.txt这里是在第八关的测试文件之上的一个更加完

整的测试，这里也就是在停止后，输出进程信息之后，使用bg命令来唤醒进程2，也就是刚才被挂起的程序，接下来继续使用Jobs命令来输出结果 结果如下



2.10这里是将后台的进程更改为前台正在运行的程序。测试文

中进程1根据&可以知道，进程1是一个后台进程。先使用fg命令将其转化为前台的一个程序，接下来停止进程1，然后打印出进程信息，这时候进程1应该是前台程序同时被挂起了，接下来使用fg命令使其继续运行，使用jobs来打印出进程信息 结果如下

```
bo@bo:~/shlab-handout$ make test09
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (22996) ./myspin 4 &
tsh> ./myspin 5
Job [2] (22998) stopped by signal 20
tsh> jobs
[1] (22996) Running ./myspin 4 &
[2] (22998) Stopped ./myspin 5
tsh> bg %2
[2] (22998) ./myspin 5
tsh> jobs
[1] (22996) Running ./myspin 4 &
[2] (22998) Running ./myspin 5
```

```
bo@bo:~/shlab-handout$ make rtest09
./sdriver.pl -t trace09.txt -s ./tshref -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (23008) ./myspin 4 &
tsh> ./myspin 5
Job [2] (23010) stopped by signal 20
tsh> jobs
[1] (23008) Running ./myspin 4 &
[2] (23010) Stopped ./myspin 5
tsh> bg %2
[2] (23010) ./myspin 5
tsh> jobs
[1] (23008) Running ./myspin 4 &
[2] (23010) Running ./myspin 5
```